Internationalization (i18n)

This project uses `i18next` (with `react-i18next` and `remix-i18next`) for internationalization. The implementation follows a dual-instance architecture with server-side and client-side i18next instances. Locale detection is handled by the multisite middleware, which resolves the locale before i18next initializes.

Quick Start Examples

For React components:

```
import { useTranslation } from "react-i18nex

function MyComponent() {
  // Get translation function for 'product'
  const { t } = useTranslation("product");
  return <h1>{t("title")}</h1>;
}
```

For everything else (loaders, actions, utilities, helpers, and tests):

```
import { getTranslation } from "@salesforce/

// Client-side or non-component code
const { t } = getTranslation();
const message = t("product:title");

// Server-side (loaders/actions) - pass the
export function loader(args: LoaderFunctionA
  const { t } = getTranslation(args.context)
  return { title: t("product:title") };
}
```

Architecture Overview

The i18n layer is split between the **SDK** and the **template**:

- **SDK (`@salesforce/storefront-next-runtime/i18n`)** – generic infrastructure:

initialization

- **Template** – translations (`src/locales/`), configuration, type augmentation, root.tsx wiring

We maintain two separate instances of i18next:

1. **Server-side instance**: Has access to all translations for the entire site
2. **Client-side instance**: Dynamically imports translations as static JavaScript chunks

Both instances support dynamic language switching at run time without page reloads.

Server-Side and Client-Side Flow

1. Server-side middleware detects the user locale and initializes i18next
2. Server has access to all translations from all locales and renders SSR content with translations
3. Client-side initializes its own i18next instance, reading the language from the HTML `lang` attribute to prevent hydration mismatches
    - The `initI18next()` function in `root.tsx` accepts an optional `{ language }` parameter to ensure consistency between server and client
4. When a translation is first requested, the client dynamically imports ALL translations for the current language
    - This triggers an HTTP request for a JavaScript chunk (e.g., `/assets/locales-en-[hash].js`)
    - The chunk is served as a static asset (pre-built, minified, and cached with long-term headers)
    - Much more efficient than an API endpoint: no server processing, CDN-friendly, immutable caching
5. All namespaces for that language are loaded and cached in memory
6. Subsequent translation requests use the cached data (no additional requests)
7. When users switch languages, the client loads the new language's translations dynamically (if not already cached) and updates the UI immediately

Configuration

Supported Languages and Currencies

```
i18n: {
    fallbackLng: 'en-GB',
    supportedLngs: ['it-IT', 'en-US', 'en-GB
}
```

2. `commerce.sites` - Per-site locale and currency configuration:

```
commerce: {
    sites: [
        {
            id: 'RefArchGlobal',
            defaultLocale: 'en-GB',
            defaultCurrency: 'GBP',
            supportedLocales: [
                { id: 'en-GB', preferredCurr
                { id: 'it-IT', preferredCurr
                { id: 'en-US', preferredCurr
            ],
            supportedCurrencies: ['EUR', 'GB
        },
    ],
}
```

The i18next middleware reads `i18n.fallbackLng` and `i18n.supportedLngs` from the config automatically. You don't need to configure the middleware separately.

must match the `id` values across all entries in `commerce.sites[].supportedLocales`.

- Each locale in `supportedLocales` has a `preferredCurrency` that matches one of the site's `supportedCurrencies`.
- Each locale in `i18n.supportedLngs` must have a corresponding translation directory under `src/locales/`.
- If you add a new language, update both `i18n.supportedLngs` and the relevant site's `supportedLocales`, and create the translation files.
- If the arrays don't match, you can get partial translations or locale/currency mismatches.

Locale Detection

Locale detection is handled by the multisite middleware, which runs before i18next initializes. The multisite middleware resolves the locale using a configurable detection chain (by default: URL path, query string, cookie, HTTP header) and passes the resolved locale to i18next via an internal request map. The i18next middleware then initializes with the resolved locale.

If no locale can be resolved from any source, the system falls back to the configured `fallbackLng`.

For details on how locale detection works and how to customize the detection order, see Configure Site and Locale Detection.

Currency System

The app supports independent locale and currency switching.

1. Locale-based currency: Each locale in `commerce.sites[].supportedLocales` has a `preferredCurrency` that's used by default.
2. Manual currency selection: Users can manually select any currency from `commerce.sites[].supportedCurrencies`, which takes precedence over the locale's preferred currency.
3. Currency priority: User's manual selection (cookie) → Locale's preferred currency → Site's default currency.

Usage Examples

In React Components

Use the `useTranslation` hook from `react-i18next`.

```
import { useTranslation } from "react-i18nex

function ProductInfo() {
  // Specify the namespace to load
  const { t } = useTranslation("product");
  // NOTE: without passing in a namespace, t
  // Since we don't have such namespace in o
  // but its autocomplete no longer works in

  return (
    <div>
      <h1>{t("title")}</h1>
      <p>{t("description")}</p>
      <button>{t("addToCart")}</button>
    </div>
  );
}
```

With multiple namespaces:

```
import { useTranslation } from "react-i18nex

function ProductPage() {
  // Load multiple namespaces at once
  const { t } = useTranslation(["home", "pro

  return (
    <div>
      <h1>{t("home:title")}</h1>
      <p>{t("product:description")}</p>
      <button>{t("product:addToCart")}</butt
    </div>
  );
}
```

With interpolation:

```
const { t } = useTranslation("cart");
const greeting = t("greeting", { name: "John
```

With pluralization:

```
const { t } = useTranslation("cart");
const text = t("summary.itemsInCart", { coun
```

Use the `getTranslation` utility for tests, utilities, or any non-React code.

```
import { getTranslation } from "@salesforce/

// In tests
describe("ActionCard", () => {
  const { t } = getTranslation();

  test("shows edit button", () => {
    render(<ActionCard onEdit={vi.fn()} />);
    const button = screen.getByRole("button"
    expect(button).toBeInTheDocument();
  });
});

// In utility functions
export function getCountryName(countryCode:
  const { t } = getTranslation();
  return t(`countries:${countryCode}.name`,
}

// In form schemas (for Zod error messages)
const schema = z.object({
  email: z.string().email(t("error:validatio
});
```

In Route Loaders and Actions (Server-Side)

Use `getTranslation` with the context parameter for server-side translations.

```
import { getTranslation, i18nextContext } fr
import type { LoaderFunctionArgs } from "rea

export function loader(args: LoaderFunctionA
  // Get translations by passing the context
  const { t } = getTranslation(args.context)
  const translatedTitle = t("product:title")

  // Get the current locale for formatting (
  const i18nextData = args.context.get(i18ne
  const locale = i18nextData?.getLocale() ??
  const date = new Date().toLocaleDateString
    year: "numeric",
    month: "2-digit",
    day: "2-digit",
  });

  return { translatedTitle, date };
}
```

In actions with error handling:

```
6
7   try {
8     // ... perform action
9     return { success: true, message: t("prod
10  } catch (error) {
11    return { success: false, message: t("err
12  }
13 }
```

File Structure

```
src/locales/
├── index.ts                # Exports all la
├── en-GB/
│   ├── index.ts            # Exports Englis
│   └── translations.json   # All English (G
├── en-US/
│   ├── index.ts            # Exports Englis
│   └── translations.json   # All English (U
└── it-IT/
    ├── index.ts            # Exports Italia
    └── translations.json   # All Italian tr

src/extensions/
├── my-extension/
│   └── locales/
│       ├── en-GB/
│       │   └── translations.json   # Extens
│       ├── en-US/
│       │   └── translations.json   # Extens
│       └── it-IT/
│           └── translations.json   # Extens
└── locales/                # Auto-generated
    ├── en-GB/
    │   └── index.ts        # Aggregated ext
    ├── en-US/
    │   └── index.ts        # Aggregated ext
    └── it-IT/
        └── index.ts        # Aggregated ext

src/components/
└── locale-switcher/
    └── index.tsx           # Client compone

src/middlewares/
└── i18next.server.ts       # Thin wrapper a

src/routes/
└── action.set-locale.ts    # Server action
```

The i18n utilities (`getTranslation` , `getLocale` , `mockI18nContext` , `createI18nMiddleware` , `initI18next`) are provided by the SDK and split across two subpaths:

- `@salesforce/storefront-next-runtime/i18n` – server-capable APIs (`getTranslation` , `getLocale` , `mockI18nContext` , `createI18nMiddleware`).

(`initI18next`). This entry pulls in `i18next-browser-languagedetector`, which has no Node support, so it must only be imported from client-side code (e.g. inside `useEffect` in `root.tsx`). Importing it from a `*.server.ts` file will fail to bundle and is blocked by ESLint.

Adding New Translations

Approach: Single JSON File Per Language

All translations are stored in a single JSON file per language with namespace-based organization.

Understanding Namespaces

i18next uses the concept of namespaces to organize translations into logical groups. In our implementation, namespaces are simply the top-level keys in each `translations.json` file. For example, `"common"`, `"product"`, `"checkout"`, and `"myNewFeature"` are all namespaces that help organize translations by feature or domain.

`src/locales/en-GB/translations.json`:

```
{
  "common": {
    "loading": "Loading",
    "product": "the product"
  },
  "product": {
    "title": "Product Details",
    "addToCart": "Add to Cart",
    "greeting": "Hello, {{name}}!",
    "itemCount": {
      "zero": "No items",
      "one": "{{count}} item",
      "other": "{{count}} items"
    }
  },
  "myNewFeature": {
    "welcome": "Welcome to the new feature"
  }
}
```

`src/locales/it-IT/translations.json`:

```
{
  "common": {
    "loading": "Caricamento",
    "product": "il prodotto"
  },
  "product": {
```

```
    "one": "{{count}} articolo",
    "other": "{{count}} articoli"
  }
},
"myNewFeature": {
  "welcome": "Benvenuto nella nuova funzio
}
}
```

Using Your New Translations

```
// In React components
const { t } = useTranslation('myNewFeature')
<p>{t('welcome')}</p>

// In non-component code
const { t } = getTranslation();
const message = t('myNewFeature:welcome');

// Simple translation
<p>{t('title')}</p>

// With interpolation
<p>{t('greeting', { name: 'John' })}</p>

// With pluralization
<p>{t('itemCount', { count: items.length })}
```

Extension Translations

Extensions can have their own translation files that are automatically discovered and integrated into the i18n system. Extension authors can keep translations co-located with their extension code.

File Structure for Extensions

Create translation files within your extension directory using this structure:

```
src/extensions/
├── my-extension/
│   ├── components/
│   ├── locales/
│   │   ├── en-GB/
│   │   │   └── translations.json
│   │   ├── en-US/
│   │   │   └── translations.json
│   │   └── it-IT/
│   │       └── translations.json
│   └── index.ts
```

Namespace Convention

- `bopis` → `extBopis`
- `my-extension` → `extMyExtension`

This convention prevents namespace collisions between extensions and core app translations.

Using Extension Translations

This example shows how to use an extension translation in a React component.

```
import { useTranslation } from "react-i18nex

export function DeliveryOptions() {
  // Use your extension's namespace
  const { t } = useTranslation("extBopis");

  return (
    <div>
      <h3>{t("deliveryOptions.title")}</h3>
      <button>{t("deliveryOptions.pickupOrDe
    </div>
  );
}
```

This example shows how to use an extension translation in non-component code.

```
import { getTranslation } from "@salesforce/

export function getDeliveryMessage() {
  const { t } = getTranslation();
  // Use namespace prefix with colon
  return t("extBopis:deliveryOptions.title")
}
```

This example shows how to use an extension translation in route loaders or actions.

```
import { getTranslation } from "@salesforce/
import type { LoaderFunctionArgs } from "rea

export function loader(args: LoaderFunctionA
  const { t } = getTranslation(args.context)
  return {
    message: t("extBopis:storePickup.title")
  };
}
```

How Locale Discovery Works

> **Important**

imported directly.

The script scans two locations to discover all supported locales:

1. Main app locales: `/src/locales/{locale}/`
2. Extension locales: `/src/extensions/{extension-name}/locales/{locale}/`

The script merges locales from both sources and generates extension-only aggregation files under `/src/extensions/locales/` for each discovered locale. This means:

- If your main app supports Italian (`it-IT`) but none of your extensions have Italian translations, an empty aggregation file is still generated for `it-IT` .
- If an extension provides translations for a locale not in the main app, those translations are still aggregated (though the main app doesn't use them unless configured).
- Extensions without a `locales` folder are automatically skipped–no error is thrown.

Example Scenario:

- Main app: `en-GB` , `en-US` , `it-IT` translations
- Extension A: `en-GB` , `en-US` translations
- Extension B: `en-GB` translations only
- Extension C: No `locales` folder

Result: Extension aggregation files generated in `/src/extensions/locales/` for `en-GB` , `en-US` , and `it-IT` :

- `en-GB/index.ts` : Contains Extension A + Extension B translations only.
- `en-US/index.ts` : Contains Extension A translations only.
- `it-IT/index.ts` : Empty (no extensions have it).

> **Note**
>
> Main app translations remain in `/src/locales/` and aren't affected by this aggregation process.

Adding Translations to an Extension

1. Create the translation files:

`US/translations.json`

```
{
  "deliveryOptions": {
    "title": "Delivery:",
    "pickupOrDelivery": {
      "shipToAddress": "Ship to Address",
      "pickUpInStore": "Pick Up in Store"
    }
  },
  "storePickup": {
    "title": "Store Pickup Location",
    "viewButton": "View",
    "closeButton": "Close"
  }
}
```

2. Translations are automatically aggregated:

When you run `pnpm dev` or `pnpm build`, the system automatically:

- Discovers all extension translation files.
- Aggregates them with the appropriate namespace.
- Makes them available to your extension code.

No manual configuration is required.

Switching Languages and Currencies at Run Time

Language Switching

Users can switch languages dynamically without reloading the page using the `LocaleSwitcher` component. The language change happens in two steps:

1. Client-side update: Immediately changes the displayed language using i18next's `changeLanguage()` method
2. Server-side persistence: Submits to a server action that sets the `lng` cookie to persist the preference across page reloads

Using the LocaleSwitcher Component

The project includes a pre-built `LocaleSwitcher` component to drop into your UI:

```
import LocaleSwitcher from "@/components/loc

```

```
    </footer>
  );
}
```

Building Your Own Language Switcher

For a custom implementation, here's how to implement language switching. In a multisite setup, the locale switcher must rebuild the current URL with the new locale prefix and trigger a full page reload to revalidate all loaders.

```
"use client";

import { useTranslation } from "react-i18nex
import { useFetcher } from "react-router";
import {
  buildUrl,
  sanitizePrefix,
  resolvePrefix,
} from "@salesforce/storefront-next-runtime/
import { useConfig } from "@salesforce/store
import { useCurrentSiteAndLocaleRef } from "

export function MyLanguageSwitcher() {
  const { i18n } = useTranslation();
  const fetcher = useFetcher();
  const config = useConfig();
  const { siteRef, localeRef } = useCurrentS

  const handleLanguageChange = async (newLoc
    const newLocaleRef = config.localeAliasM

    // Strip current prefix, rebuild with ne
    const currentPrefix = config.url?.prefix
      ? resolvePrefix(config.url.prefix, { s
      : "";
    const barePath = sanitizePrefix(location

    const pathname = buildUrl({
      to: barePath,
      urlConfig: config.url,
      params: { siteId: siteRef, localeId: n
    });

    // Step 1: Change language client-side f
    await i18n.changeLanguage(newLocale);

    // Step 2: Persist to server cookie and
    const formData = new FormData();
    formData.append("locale", newLocale);
    formData.append("pathname", pathname);
    await fetcher.submit(formData, {
      method: "POST",
      action: "/action/set-locale",
    });

    // Step 3: Full page reload to revalidat
    window.location.href = pathname;
  };

  return (
```

```
      ))}
    </select>
  );
}
```

How It Works

The `/action/set-locale` server action, which is located at `src/routes/action.set-locale.ts`, receives the POST request and sets the locale cookie using the multisite cookie from router context. It then redirects to the provided pathname, which includes the new locale in the URL prefix.

```
import { redirect, type ActionFunction } fro
import { getMultiSiteCookies } from '@salesf

export const action: ActionFunction = async
    const formData = await request.formData(
    const locale = formData.get('locale') as
    const pathname = formData.get('pathname'

    if (!locale) {
        throw new Response('Locale is requir
    }

    const cookies = getMultiSiteCookies(cont
    if (!cookies) {
        throw new Response('Site and locale
    }

    const cookieHeader = await cookies.local

    return redirect(pathname || '/', {
        headers: {
            'Set-Cookie': cookieHeader,
        },
    });
};
```

Key Points

- The client-side language change via `i18n.changeLanguage()` provides an immediate UX update.
- In a multisite setup, locale switching triggers a full page reload to revalidate all loaders with the new locale and update the URL prefix.
- The preference persists across sessions via the locale cookie (managed by the multisite middleware).
- All client-side translations are loaded as static assets (one JavaScript chunk per language).
- Switching languages triggers the dynamic import of the new language's translations if not already loaded.

1. Server submits a server action.
2. Middlewares (client and server) run to update the latest currency into context.
3. Calls the `updateBasket` endpoint in SCAPI to update the currency accordingly.
4. Loader function revalidates and updates the UI to reflect the selected currency.

Using the CurrencySwitcher Component

```
1  import CurrencySwitcher from "@/components/c
2  import LocaleSwitcher from "@/components/loc
3
4  export function Footer() {
5    return (
6      <footer>
7        <div>
8          <h3>Language</h3>
9          <LocaleSwitcher />
10       </div>
11       <div>
12         <h3>Currency</h3>
13         <CurrencySwitcher />
14       </div>
15     </footer>
16   );
17 }
```

Key Points

- Currency selection is independent of locale
- Manual currency selection takes precedence over locale's preferred currency
- The preference persists across locale changes
- Falls back to locale's preferred currency if no manual selection is made

Best Practices

1. Namespace by Route/Feature: Organize translations by feature area (for example, `product`, `checkout`, `account`).
2. Use the right tool:
    - React components: Use `useTranslation()` hook
    - Everything else: Use `getTranslation()` function
        - Non-component code (tests, utilities, schemas): `getTranslation()`
        - Server-side loaders/actions: `getTranslation(context)`

5. Pluralization: Use nested objects with `zero`, `one`, `other` keys for count-based translations.
6. Lazy Loading: Client-side translations are loaded on-demand when first requested.
7. Fallback Chain: Missing translations fall back to the configured `fallbackLng`.

Type Safety

The project is configured for type-safe translations. TypeScript autocompletes available keys and warn about missing translations:

```
1  // ✅ TypeScript knows these keys exist
2  const { t } = useTranslation("product");
3  t("title");
4  t("addToCart");
5
6  // With namespace prefix in non-component co
7  const { t } = getTranslation();
8  t("product:title");
9  t("cart:empty.title");
10
11 // ❌ TypeScript will warn about this
12 t("nonexistent.key");
```

Type definitions are generated from the English (GB) locale. In `src/middlewares/i18next.server.ts`:

```
1  declare module "i18next" {
2    interface CustomTypeOptions {
3      resources: typeof resources["en-GB"]; //
4    }
5  }
```